2.1 Computer Architecture

central processing unit (CPU)

The circuitry in a computer that controls the manipulation of data is called the central processing unit (CPU), often referred to simply as the processor.

计算机中负责**控制数据处理（运算和操作）**的电路称为中央处理器（CPU），也常简称为处理器。

Motherboard(主板)

These CPUs plug into a socket mounted on the machine's main circuit board, called the motherboard.

CPU 通过引脚插入到主电路板（即主板，motherboard）上的插槽中。

Mobile Internet Devices (MID)

In smartphones, mini notebooks, and other Mobile Internet Devices (MID), CPUs are about half the size of a postage stamp.

Microprocessors

Due to their small size, these processors are called microprocessors.

CPU Basics

arithmetic/logic unit (ALU)

The arithmetic/logic unit contains the circuitry that performs operations on data, such as addition and subtraction.

control unit

The control unit contains the circuitry for coordinating the machine's activities.

register unit

The register unit contains data storage cells, called registers, used for temporary storage of information within the CPU.

registers

-general-purpose registers

General-purpose registers serve as temporary holding places for data being manipulated by the CPU.存输入数据 存运算结果

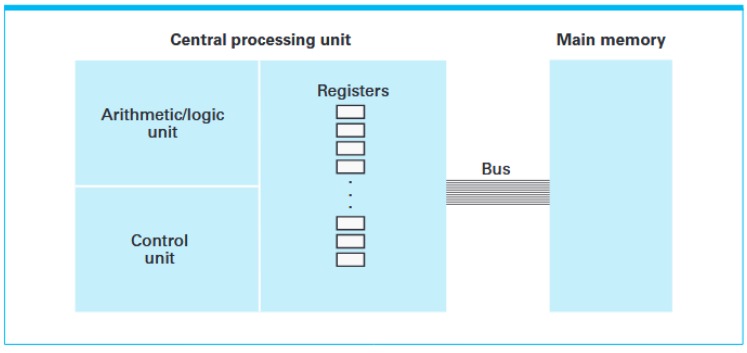
-special-purpose registers

Special-purpose registers are used for specific tasks within the CPU.

Bus

A bus is a collection of wires that connects the CPU and main memory for transferring bit patterns.

Figure 2.1 CPU and main memory connected via a bus



How CPU Processes Data

To perform an operation on data stored in main memory, the control unit transfers the data into general-purpose registers, instructs the arithmetic/logic unit, and stores the result. 为了对主存中的数据进行操作，控制单元会将数据传入通用寄存器，指示 ALU 进行运算，并存储结果。

Memory Read / Write

Reading Data

The control unit sends the address of the memory cell.

The control unit sends a read signal.

The data is transferred from memory to a register.

Writing Data

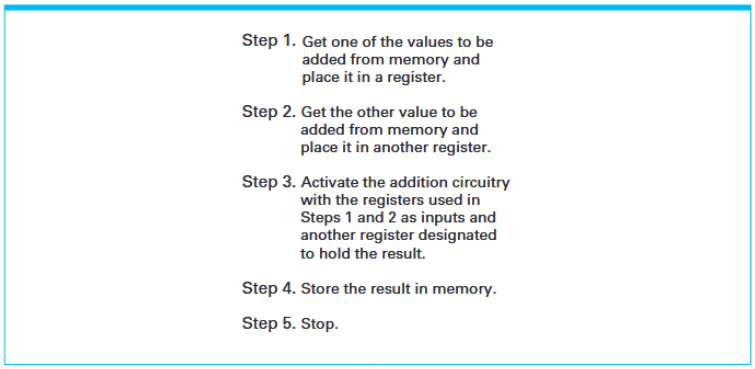
The control unit sends the address.

The control unit sends the data.

The control unit sends a write signal.

Example: Adding Two Numbers

Figure 2.2 Adding values stored in memory



The Stored-Program Concept

stored-program concept

The idea of storing a program in main memory is called the stored-program concept.

The control unit can fetch the program from memory, decode the instructions, and execute them.

Memory Hierarchy（内存层次结构）

-Registers

Registers hold data that is immediately needed for current operations.

-Main Memory

Main memory holds data that will be needed in the near future.

-Mass Storage

Mass storage holds data that is not needed immediately.

-Cache Memory

Cache memory is a portion of high-speed memory located within the CPU.

Cache memory stores copies(副本) of frequently used data from main memory.

Data flows from main memory → cache memory → registers → ALU.

1. What sequence of events do you think would be required to move the contents of one memory cell in a computer to another memory cell?

The control unit sends the address of the source memory cell along with a read signal to load

the data into a register.
Then it sends the address of the destination memory cell, the data, and a write signal to store the data in that cell.

2. What information must the CPU supply to the main memory circuitry to write a value into a memory cell?
Address
Data
a write signal

3. Mass storage, main memory, and general purpose registers are all storage systems. What is the difference in their use?
Mass storage holds data that is not needed immediately.
Main memory holds data that will be needed in the near future.
Registers hold data that is immediately needed for current operations.

2.2 Machine Language

machine language is a collection of instructions encoded as bit patterns that a CPU can recognize and execute.

machine instruction A machine instruction is a single instruction expressed in machine language.

The Instruction Repertoire（指令集）

The instruction repertoire is the set of all machine instructions that a CPU can decode and execute.

reduced instruction set computer (RISC) 精简指令集

A reduced instruction set computer (RISC) is a CPU designed to execute a minimal set of simple instructions.

Fixed-Length

Example: ARM (Advanced RISC Machine) processors are examples of RISC architecture and are widely used in mobile devices.

complex instruction set computer (CISC) 复杂指令集计算机

A complex instruction set computer (CISC) is a CPU designed to execute a large number of complex instructions.

Variable-Length

Example: Intel processors are examples of CISC architecture.

Instruction Categories

Machine instructions can be divided into three groups: data transfer, arithmetic/logic, and control.

Data Transfer

The data transfer group consists of instructions that move (copy) data from one location to another.

A **LOAD** instruction transfers data from main memory to a register.

A **STORE** instruction transfers data from a register to main memory.

I/O instructions

I/O instructions handle communication between the computer and external devices such as keyboards, displays, and disks.

Arithmetic/Logic

The arithmetic/logic group consists of instructions that perform operations in the arithmetic/logic unit (ALU).

These operations include arithmetic operations (such as addition) and logical operations (such as AND, OR, XOR).

SHIFT operations move bits left or right and discard overflow bits, while ROTATE operations reuse overflow bits. SHIFT 操作将位左移或右移并丢弃溢出位，ROTATE 操作会将溢出位循环使用。

Control

JUMP / BRANCH

Jump (or branch) instructions tell the CPU to execute an instruction other than the next one in sequence.

unconditional jumps

An unconditional jump always transfers control to another instruction.

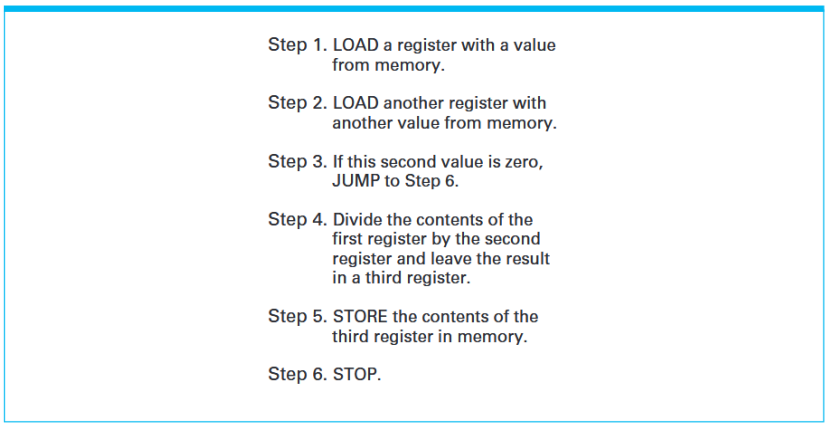
“Go to Step 5”

conditional jumps

A conditional jump transfers control only if a specified condition is satisfied.

“If the value is 0, go to Step 5”

Figure 2.3 Dividing values stored in memory



An Illustrative Machine Language

Basic Structure: The illustrative machine has 16 general-purpose registers and 256 memory cells, each storing 8 bits.

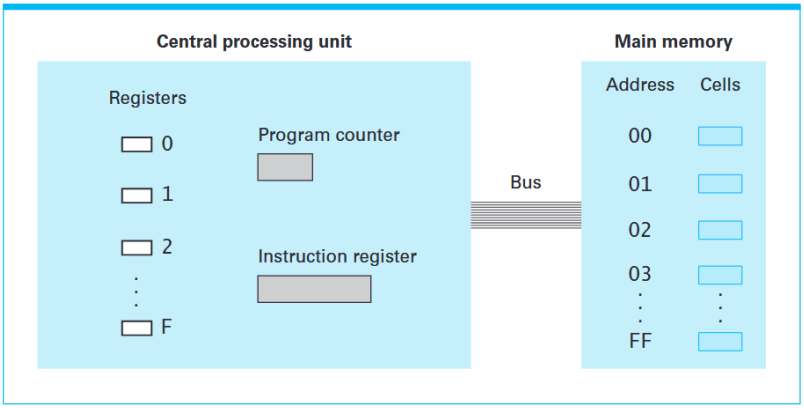
Addressing System: Registers are labeled 0 to F, and memory cells are addressed from 00 to FF using hexadecimal notation.

Instruction Structure Each machine instruction consists of two parts: the op-code field and the operand field.

op-code（操作码） The op-code indicates which operation the CPU should perform.

operand（操作数） The operand field provides detailed information about the operation.

Figure 2.4 The architecture of the machine described in Appendix C



Example 1: STORE Instruction

Figure 2.5 The composition of an instruction for the machine in Appendix C

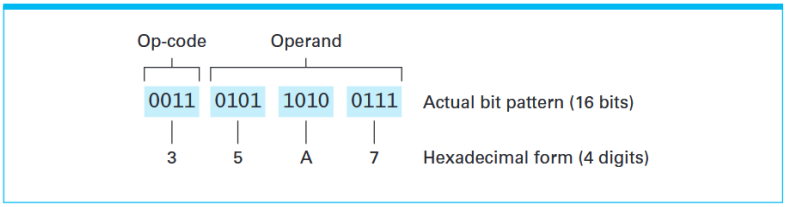
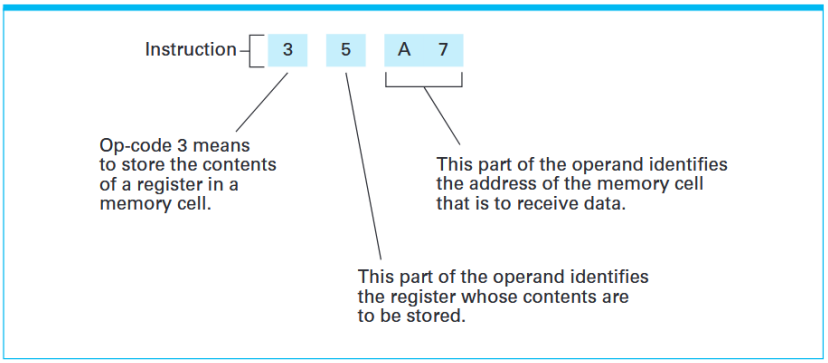


Figure 2.6 Decoding the instruction 35A7



3 → op-code（STORE）

5 → 寄存器编号

A7 → 内存地址

LOAD Instructions

Op-code 1 loads a register with data from a memory cell. （从内存加载）

Op-code 2 loads a register with a specific value. （加载常数）

ADD Instructions

The machine has different ADD instructions for two's complement and floating-point numbers.

Memory Design Insight

Because 8 bits are used for addressing, the machine can access 2^8 = 256 memory cells.

Figure 2.7 An encoded version of the instructions in Figure 2.2

Encoded instructions	Translation
156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
C000	Halt.

1. Why might the term move be considered an incorrect name for the operation of moving data from one location in a machine to another?

The term "move" is misleading because the data is usually copied rather than removed from its original location.

2. In the text, JUMP instructions were expressed by identifying the destination explicitly

by stating the name (or step number) of the destination within the JUMP instruction (for example, "Jump to Step 6"). A drawback of this technique is that if an instruction name (number) is later changed, we must be sure to find all jumps to that instruction and change that name also. Describe another way of expressing a JUMP instruction so that the name of the destination is not explicitly stated.

A JUMP instruction can specify the destination by using a memory address instead of a step number.

地址 指令

00 Step 1

01 Step 2

02 Step 3

03 Step 4

04 Step 5

05 Step 6

"Jump to Step 6" actually means "Jump to address 05".

3. Is the instruction "If 0 equals 0, then jump to Step 7" a conditional or unconditional jump? Explain your answer.

unconditional jump. because "0 equals 0" always be true.

4. Write the example program in Figure 2.7 in actual bit patterns.

156C → 0001 0101 0110 1100
166D → 0001 0110 0110 1101
5056 → 0101 0000 0101 0110
306E → 0011 0000 0110 1110
C000 → 1100 0000 0000 0000

5. The following are instructions written in the machine language described in Appendix C. Rewrite them in English.

a. 368A b. BADE c. 803C d. 40F4

a. Store the contents of register 6 in the memory cell at 8E.

b. first compare the contents of register A with the contents of register 0. If the two were equal, the pattern DE would be placed in the program counter so that the next instruction executed would be the one located at that memory address. Otherwise, nothing would be done and program execution would continue in its normal sequence.

c. 803C would cause the result of ANDing the contents of registers 3 and C to be placed in register 0.

4. 40F4 would cause the contents of register F to be copied into register 4.

6. What is the difference between the instructions 15AB and 25AB in the machine language of Appendix C?

15AB loads register 5 with the contents of memory cell AB.

25AB loads register 5 with the value AB.

7. Here are some instructions in English. Translate each of them into the machine language of Appendix C.

a. LOAD register number 3 with the hexadecimal value 56.

b. ROTATE register number 5 three bits to the right.

c. AND the contents of register A with the contents of register 5 and leave the result in register 0.

a. 2356

b. A503 [A][register][direction+bits]

c. 80A5 [result][reg1][reg2]

Op-code	Operand	Description
1	RXY	LOAD the register R with the bit pattern found in the memory cell whose address is XY. *Example:* 14A3 would cause the contents of the memory cell located at address A3 to be placed in register 4.
2	RXY	LOAD the register R with the bit pattern XY. *Example:* 20A3 would cause the value A3 to be placed in register 0.

Op-code	Operand	Description
3	RXY	STORE the bit pattern found in register R in the memory cell whose address is XY. *Example:* 35B1 would cause the contents of register 5 to be placed in the memory cell whose address is B1.
4	0RS	MOVE the bit pattern found in register R to register S. *Example:* 40A4 would cause the contents of register A to be copied into register 4.
5	RST	ADD the bit patterns in registers S and T as though they were two's complement representations and leave the result in register R. *Example:* 5726 would cause the binary values in registers 2 and 6 to be added and the sum placed in register 7.
6	RST	ADD the bit patterns in registers S and T as though they represented values in floating-point notation and leave the floating-point result in register R. *Example:* 634E would cause the values in registers 4 and E to be added as floating-point values and the result to be placed in register 3.
7	RST	OR the bit patterns in registers S and T and place the result in register R. *Example:* 7CB4 would cause the result of ORing the contents of registers B and 4 to be placed in register C.
8	RST	AND the bit patterns in registers S and T and place the result in register R. *Example:* 8045 would cause the result of ANDing the contents of registers 4 and 5 to be placed in register 0.

9	RST	EXCLUSIVE OR the bit patterns in registers S and T and place the result in register R. *Example:* 95F3 would cause the result of EXCLUSIVE ORing the contents of registers F and 3 to be placed in register 5.
A	R0X	ROTATE the bit pattern in register R one bit to the right X times. Each time place the bit that started at the low-order end at the high-order end. *Example:* A403 would cause the contents of register 4 to be rotated 3 bits to the right in a circular fashion.
B	RXY	JUMP to the instruction located in the memory cell at address XY if the bit pattern in register R is equal to the bit pattern in register number 0. Otherwise, continue with the normal sequence of execution. (The jump is implemented by copying XY into the program counter during the execute phase.) *Example:* B43C would first compare the contents of register 4 with the contents of register 0. If the two were equal, the pattern 3C would be placed in the program counter so that the next instruction executed would be the one located at that memory address. Otherwise, nothing would be done and program execution would continue in its normal sequence.
C	000	HALT execution. *Example:* C000 would cause program execution to stop.

2.3 Program Execution

A computer executes a program by fetching instructions from memory, decoding them, and executing them.

取指令 → 译码 → 执行

Instructions are normally executed in the order they are stored in memory unless a JUMP instruction changes the order.

instruction register（指令寄存器）

The instruction register holds the instruction currently being executed.

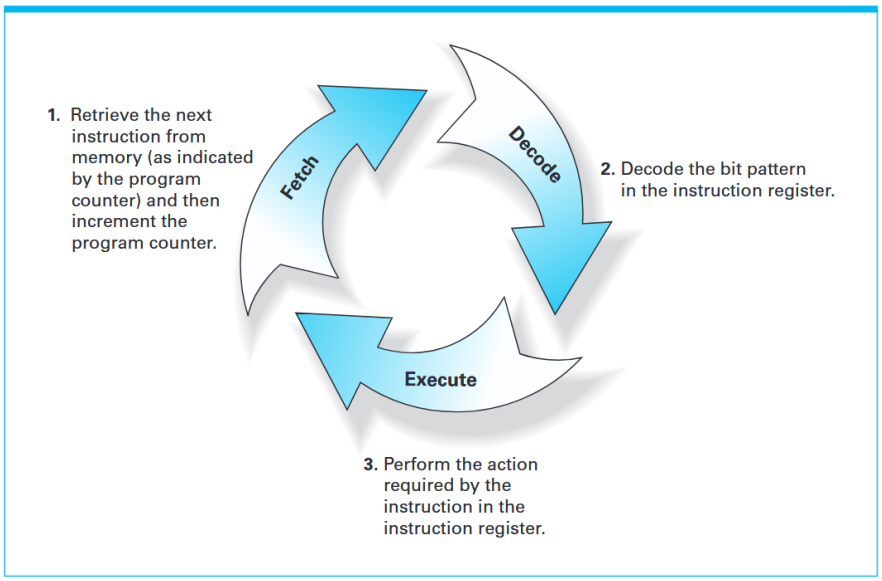
program counter（程序计数器）

The program counter contains the address of the next instruction to be executed.

machine cycle 机器周期

The CPU repeatedly follows a three-step process called the machine cycle: fetch, decode, and execute.

Figure 2.8 The machine cycle

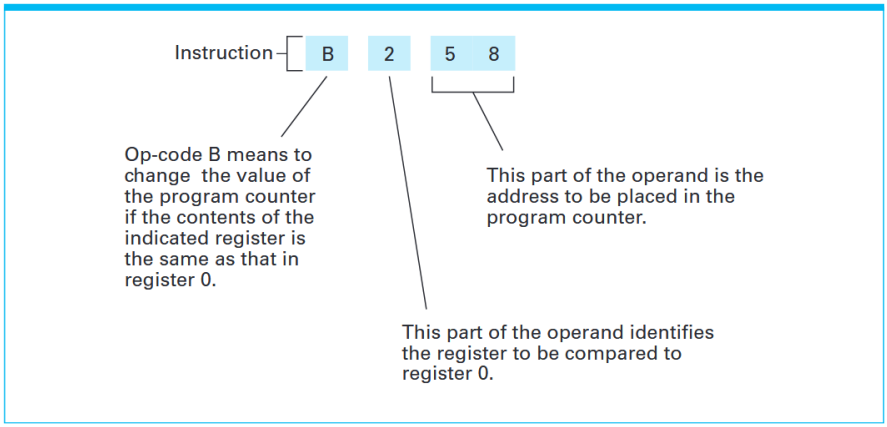


JUMP Instruction

A JUMP instruction changes the program counter to a new address.

Example: If register 2 equals register 0, the program counter is set to address 58.

Figure 2.9 Decoding the instruction B258



B0XY

If both registers are the same (e.g., register 0 and register 0), the condition is always true.

Comparing Computer Power（计算机性能比较）

Clock speed is often used to compare the performance of computers.

Clock

The clock is a circuit (oscillator 振荡器) that generates pulses（脉冲） to coordinate the activities of a computer.

Each pulse of the clock helps control the execution of machine cycles.

The faster the clock generates pulses, the faster the machine executes instructions.

Clock Speed（时钟频率）

Clock speed is measured in hertz (Hz), where 1 Hz equals one cycle per second.

MHz means one million Hz, and GHz means one billion Hz.

Clock speed alone is not a reliable way to compare different computers. Different CPUs may perform different amounts of work in one clock cycle.

Benchmarking

Benchmarking is the process of comparing computer performance by running the same program on different machines.

A benchmark is a program used to evaluate performance.

An Example of Program Execution

Figure 2.10 The program from Figure 2.7 stored in main memory ready for execution

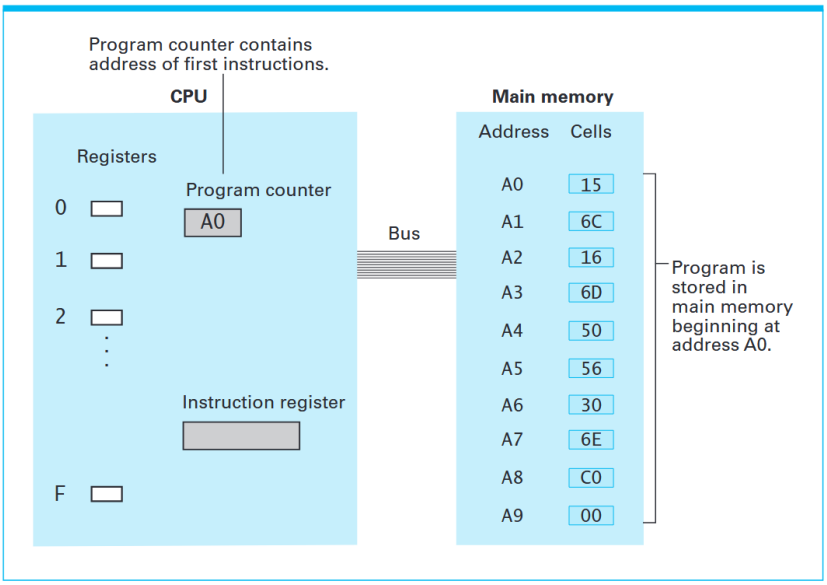
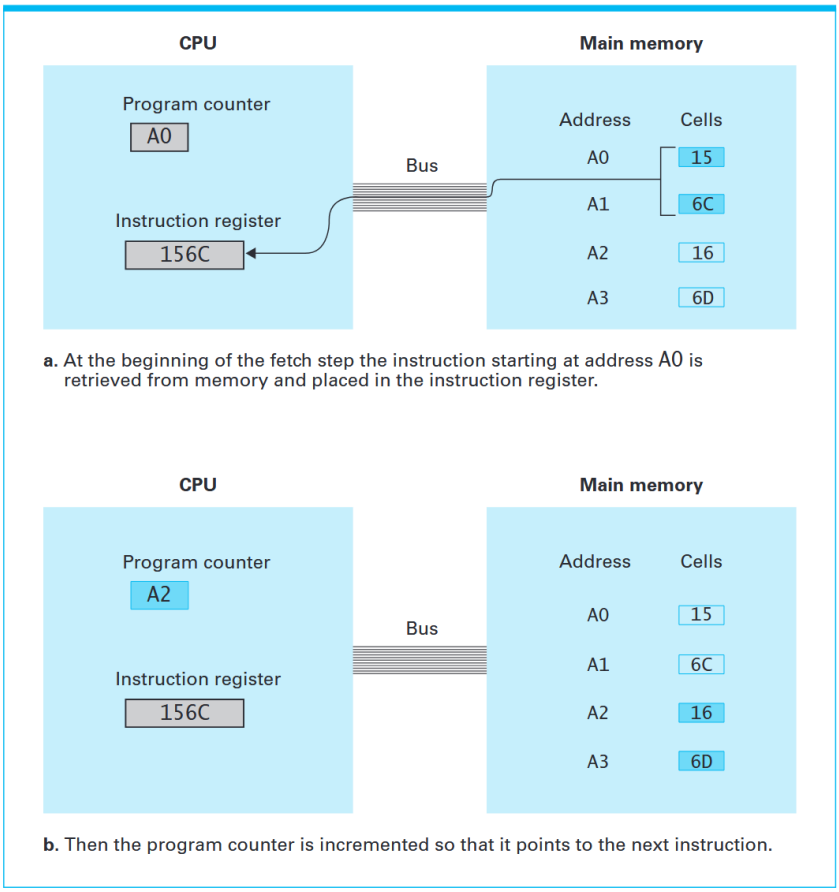
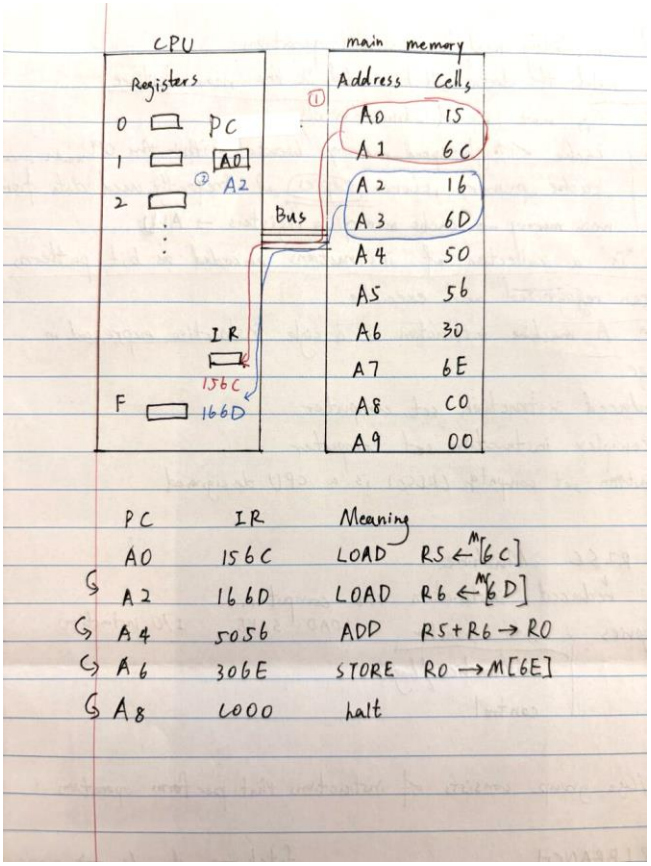


Figure 2.11 Performing the fetch step of the machine cycle





Programs Versus Data

Because programs and data are stored in the same format, the CPU cannot distinguish between them, which allows programs to manipulate other programs.

同一段东西：可以是“数字”也可以是“指令”

完全取决于：CPU 有没有把它当作指令去执行

Programs can modify their own instructions.

Programs can create other programs.

1. Suppose the memory cells from addresses 00 to 05 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address Contents

00 14

01 02

02 34

03 17

04 C0

05 00

If we start the machine with its program counter containing 00, what bit pattern is in the memory cell whose address is hexadecimal 17 when the machine halts?

IR meaning

1402 load R4 <- M [02]

3417 store R4 -> M [17]

C000 halt

Result: M [02] =34

2. Suppose the memory cells at addresses B0 to B8 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address Contents

B0 13

B1 B8

B2 A3

B3 02

B4 33

B5 B8

B6 C0

B7 00

B8 0F

a. If the program counter starts at B0, what bit pattern is in register number 3 after the first instruction has been executed?

13B8 LOAD R3 <- M[B8]

M[B8] = 0F

b. What bit pattern is in memory cell B8 when the halt instruction is executed?

13B8 LOAD R3 <- M[B8]

A302 the contents of R3 to be rotated 2 bits to the right in a circular fashion

33B8 STORE R3 -> M [B8]
C000 halt

0F 0000 1111
1100 0011

3. Suppose the memory cells at addresses A4 to B1 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
A4	**20**
A5	**00**
A6	**21**
A7	**03**
A8	**22**
A9	**01**
AA	**B1**
AB	**B0**
AC	**50**
AD	**02**
AE	**B0**
AF	**AA**
B0	**C0**
B1	**00**

When answering the following questions, assume that the machine is started with its program counter containing A4.

a. What is in register 0 the first time the instruction at address AA is executed?

2000 LOAD R0 <- 00
2103 LOAD R1 <- 03
2201 LOAD R2 <- 01
B1B0 If R1 == R0, jump to B0
R0 = 00

b. What is in register 0 the second time the instruction at address AA is executed?

5020 R0+R2 -> R0
B0AA If R0 == R0, jump to AA
B1B0 If R1 == R0, jump to B0
R0 = 01

c. How many times is the instruction at address AA executed before the machine halts?

5020 R0+R2 -> R0
B0AA If R0 == R0, jump to AA
B1B0 If R1 == R0, jump to B0
R0 = 02
5020 R0+R2 -> R0
B0AA If R0 == R0, jump to AA
B1B0 If R1 == R0, jump to B0

R0 = 3
C000 halt

4. Suppose the memory cells at addresses F0 to F9 in the machine described in Appendix C contain the (hexadecimal) bit patterns described in the following table:

Address	Contents
F0	**20**
F1	**C0**
F2	**30**
F3	**F8**
F4	**20**
F5	**00**
F6	**30**
F7	**F9**
F8	**FF** C0
F9	**FF** 00

If we start the machine with its program counter containing F0, what does the machine do when it reaches the instruction at address F8?

20C0 LOAD R0 <- C0
30F8 STORE R0 -> M [F8]
2000 LOAD R0 <- 00
30F9 STORE R0 -> M [F9]
C000 HALT

2.4 Arithmetic/Logic Instructions

Logic Operations

bitwise operations

Bitwise operations apply logic operations (AND, OR, XOR) to each pair of corresponding bits in two bit strings.

-AND Operation

10011010
11001001

10001000

-OR Operation

10011010
11001001

11011011

-XOR Operation

10011010
11001001

01010011

masking（掩码操作）

Masking is the process of using a mask to control which bits of a bit string are affected by an operation.

Mask（掩码）

A mask is a bit pattern used to select or modify specific bits in another bit string.

-Masking with AND（用 AND 做掩码）

In AND masking, bits corresponding to 0 in the mask become 0 in the result.

Mask: 00001111
Data: 10101010
Result: 00001010

-Masking with OR（用 OR 做掩码）

In OR masking, bits corresponding to 1 in the mask become 1 in the result.

Mask: 11110000
Data: 10101010
Result: 11111010

bit map（位图）

A bit map is a sequence of bits in which each bit represents the presence or absence of a particular object.

-Bit Checking（检查某一位）

To check a specific bit, we AND the bit string with a mask that has 1 in that position.

Mask: 00100000

-Bit Modification（修改某一位）

变成 0 Use AND with a mask containing 0 at that position.

变成 1 Use OR with a mask containing 1 at that position.

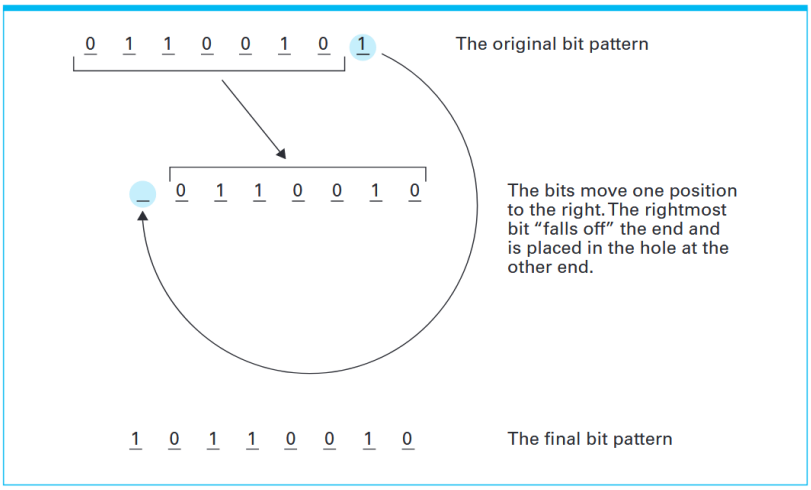
Rotation and Shift Operations

Rotation and shift operations move bits within a register.

Circular Shift / Rotation（循环移位 / 旋转）

A circular shift, also called a rotation, moves bits so that the bits that fall off one end re-enter at the other end.

Figure 2.12 Rotating the bit pattern 65 (hexadecimal) one bit to the right



logical shift（逻辑移位）

In a logical shift, bits that fall off are discarded, and empty positions are filled with 0s.

Shifting left corresponds to multiplication by 2. 左移相当于乘以 2。

Shifting right corresponds to division by 2. 右移相当于除以 2。

arithmetic shifts（算术移位）

Arithmetic shifts preserve the sign bit when shifting. 算术移位会保留符号位（最高位）。

The leftmost bit (sign bit) remains unchanged in right shifts.

Used when dealing with signed numbers (two's complement).

Arithmetic Operations

Arithmetic operations include addition, subtraction, multiplication, and division.

Subtraction can be simulated by addition and negation.

1. Perform the indicated operations.

位数不一样 → 左边补 0 → 对齐 → 再计算

a.	01001011	b.	100000011	c.	11111111
	AND 10101011		AND 11101100		AND 00101101
	00001011		000000000		00101101
d.	01001011	e.	10000011	f.	11111111
	OR 10101011		OR 11101100		OR 00101101
	11101011		11101111		11111111
g.	01001011	h.	100000011	i.	11111111
	XOR 10101011		XOR 11101100		XOR 00101101
	11100000		111101111		11010010

2. Suppose you want to isolate the middle four bits of a byte by placing 0s in the other four bits without disturbing the middle four bits. What mask must you use together with what operation?

00 ---- 00

00 1111 00 AND

Use the mask 00111100 with the AND operation.

3. Suppose you want to complement(取反) the four middle bits of a byte while leaving the other four bits undisturbed. What mask must you use together with what operation?

-- 反反反反 --

00 1111 00

Use the mask 00111100 with the XOR operation.

4. a. Suppose you XOR the first two bits of a string of bits and then continue down the string by successively XORing each result with the next bit in the string.
How is your result related to the number of 1s appearing in the string?

Even（偶） 1100 -> 0

Odd（奇） 1011 -> 1

The result is 1 if the number of 1s is odd, and 0 if it is even.

b. How does this problem relate to determining what the appropriate parity bit（奇偶校验位） should be when encoding a message?

A parity bit is an extra bit added to ensure the total number of 1s is either even or odd.

5. It is often convenient to use a logical operation in place of a numeric one. For example, the logical operation AND combines two bits in the same manner as multiplication. Which logical operation is almost the same as adding two bits, and what goes wrong in this case?

0 + 0 = 0 0 XOR 0 = 0

0 + 1 = 1 0 XOR 1 = 1

1 + 0 = 1 1 XOR 0 = 1

1 + 1 = 10 1 XOR 1 = 0 ❗

The XOR operation is almost the same as binary addition.

XOR does not account for the carry bit.

6. What logical operation together with what mask can you use to change ASCII codes of lowercase letters to uppercase? What about uppercase to lowercase?

a = 0110 0001

A = 0100 0001

小写 → 大写 1101 1111 And

大写 → 小写 0010 0000 OR

7. What is the result of performing a three bit right circular shift on the fol lowing bit strings:

a. 01101010 -> 01001101

b. 00001111 -> 11100001
c. 01111111 -> 11101111

8. What is the result of performing a one bit left circular shift on the fol lowing bytes represented in hexadecimal notation? Give your answer in hexadecimal form.
a. AB 1010 1011 -> 0101 0111 57
b. 5C 0101 1100 -> 1011 1000 B8
c. B7 1011 0111 -> 0110 1111 6F
d. 35 0011 0101 -> 0110 1010 6A

9. A right circular shift of three bits on a string of eight bits is equivalent to a left circular shift of how many bits?
12345678
Right 67812345
Left 5bits

10. What bit pattern represents the sum of 01101010 and 11001100 if the pat terns represent values stored in two's complement notation?
01101010
+ 11001100
---
1 00110110
The result is 00110110.
What if the patterns represent values stored in the floating point format discussed in Chapter 1?
1 bit sign（符号位）
3 bits exponent（指数）
4 bits mantissa（尾数）

0 | 110 | 1010
sign = 0 → 正
exponent = 110_2 = 6 → 实际指数 = 6 - 3 = 3
mantissa = 1010 → 规范化为：1.1010
1.1010 × 2^3

1 | 100 | 1100
sign = 1 → 负
exponent = 100_2 = 4 → 实际指数 = 4 - 3 = 1
mantissa = 1100 →1.1100
- (1.1100 × 2^1)
1.1100 × 2^1→ 0.011100 × 2^3

1.1010
- 0.0111

1.0011

结果：1.0011 × 2^3

11. Using the machine language of Appendix C, write a program that places a 1 in the most significant bit of the memory cell whose address is A7 without modifying the remaining bits in the cell.
使内存地址为 A7 的存储单元的最高位（most significant bit）变为 1，并且不改变该单元中其余的位。
1--- ----
MASK: 1000 0000 OR
1000 0000 = 8 0（hex）
① 从内存读数据
LOAD R1 ← M[A7] 11A7
② 准备 mask
LOAD R2 ← 80 2280
③ OR 操作
R1 ← R1 OR R2 7112
④ 写回内存
STORE R1 → M[A7] 31A7
⑤ HALT C000

12. Using the machine language of Appendix C, write a program that copies the middle four bits from memory cell E0 into the least significant four bits of memory cell E1, while placing 0s in the most significant four bits of the cell at location E1.
1）从内存读数据
LOAD R1 ← M[E0] 11E0
2）往右移动两位
Right rotate R1 A102
3）把左边四位全变 0
mask: 00001111AND
准备 mask
LOAD R2 ← 0F 220F
4）AND 操作
R1 ← R1 AND R2 8112
5）写回内存
STORE R1 → M[E1] 31E1

2.5 Communicating with Other Devices

The CPU and main memory form the core of a computer system.

Peripheral devices(外设) communicate with the computer through intermediary components (中间设备).

The Role of Controllers

Controller

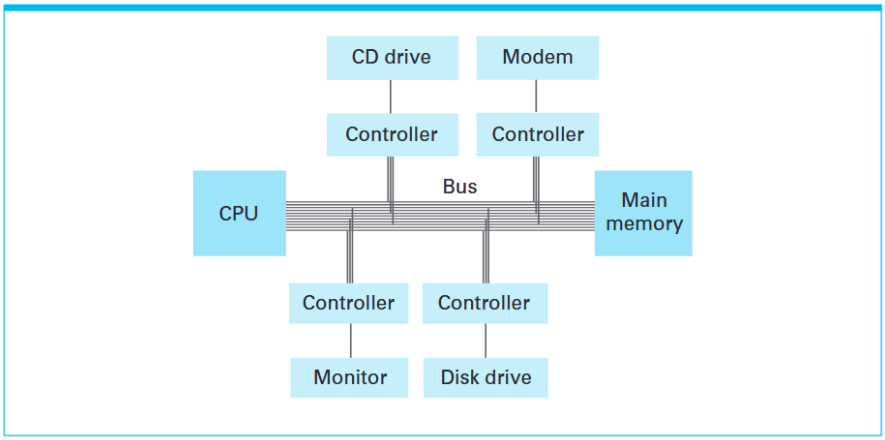
A controller is an intermediary device that manages communication between the computer and peripheral devices.

A controller may be built into the motherboard or exist as an external circuit board.

Some controllers are small computers with their own memory and CPU.

A controller translates data between the computer and the peripheral device.

Figure 2.13 Controllers attached to a machine's bus



Port

[CPU] ↔ [Controller] ↔ [Port] ↔ [Device]

A port is a connector through which external devices are attached to a computer.

USB/ HDMI

universal serial bus (USB) （通用串行总线）

USB is a standard interface that allows one controller to handle many types of devices.

A single USB controller can communicate with devices like mice, printers, cameras, and smartphones.

FireWire

FireWire is another standardized interface for connecting high-speed devices.

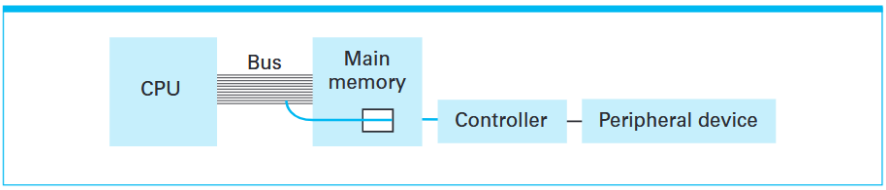
Feature	USB	FireWire
Cost	Low	Higher
Speed	Moderate (USB 2.0), High (USB 3.0)	Higher
Usage	Mass market	High-performance devices

memory-mapped I/O

Memory-mapped I/O is a technique where I/O devices are treated as memory locations.

Each controller is assigned a unique set of memory addresses. Main memory ignores these addresses, but controllers respond to them.

Figure 2.14 A conceptual representation of memory-mapped I/O



对 CPU 来说：没有区别还是普通 LOAD / STORE
对系统来说：实际是在做 I/O I/O device = memory location（逻辑上）

Direct Memory Access

direct memory access (DMA)

-Direct Memory Access (DMA) is the ability of a controller to access main memory directly without CPU intervention. 指控制器可以不通过 CPU，直接访问主存。

- A controller can use the bus when the CPU is not using it. This improves system performance by allowing parallel activities. “并行工作”

- Key Advantage of DMA

DMA allows the CPU and controller to work simultaneously. This prevents the CPU from being idle during slow I/O operations.

- Drawback of DMA

DMA complicates communication on the system bus.

von Neumann Architecture（冯·诺依曼结构）

The von Neumann architecture is a computer design in which the CPU fetches instructions and data from memory over a single bus. 指令和数据都在主存 通过同一条总线传输

von Neumann bottleneck（冯诺依曼瓶颈）

The von Neumann bottleneck refers to the limitation caused by the shared bus between CPU and memory.

CPU 和所有设备共用一条总线 → 造成速度限制

Handshaking

Handshaking

Handshaking is a two-way communication process between the computer and a peripheral device to coordinate data transfer.

Data transfer between components is rarely one-way.

-Example: A printer not only receives data but also sends data back to the computer. The computer can send data faster than the printer can process it. Without coordination, data may be lost.

Handshaking is a continuous dialogue between the computer and the device.

status word（状态字）

A status word is a bit pattern generated by a peripheral device to indicate its current condition. The status word is a bit map in which each bit represents a specific condition.

1. The device generates a status word.
2. The controller receives the status word.
3. The CPU or controller decides whether to send data.
4. Data transfer occurs only if the device is ready.

Popular Communication Media

Communication between computing devices occurs over two types of paths: parallel （并行）

and seria l（串行）.

parallel communication（并行通信）

Parallel communication transfers multiple signals at the same time over separate lines.

serial communication（串行通信）

Serial communication transfers signals one after another over a single line.

- Short Distance: USB FireWire
- Medium Distance: Ethernet
- Long Distance: Traditional telephone lines

modem（调制解调器）

A modem (modulator-demodulator) converts digital data into audible tones and back.

数字信号 → 模拟声音（调制）

声音 → 数字信号（解调）

It enables digital data to be transmitted over telephone lines.

DSL (Digital Subscriber Line)（数字用户线路）

DSL is a technology that provides faster data transmission over traditional telephone lines.

DSL uses higher frequency ranges for data while reserving lower frequencies for voice.

高频 → 数据

低频 → 语音

Fiber Optic Communication（光纤通信）

Fiber optic lines support digital communication more efficiently than telephone lines.

Cable modems（有线调制解调器）

Cable modems convert digital data for transmission over cable television systems.

Satellite Communication（卫星通信）

Satellite links use high-frequency radio signals for long-distance communication.

They provide access in remote areas without wired infrastructure.

Communication Rates

bits per second (bps)

Kbps = 1,000 bps

Mbps = 10^6 bps

Gbps = 10^9 bps

Multiplexing（多路复用）

Multiplexing is the technique of combining multiple signals into a single communication path.

bandwidth（带宽）

Bandwidth is often used to describe the maximum data transfer rate of a communication path.

带宽 = 速度 + 容量（能力）

broadband（宽带）

Broadband refers to a communication system with high bandwidth. 具有高带宽的通信系统

宽带 = 高速 + 高容量

1. Assume that the machine described in Appendix C uses memory mapped I/O and that the address B5 is the location within the printer port to which data to be printed should be sent.

a. If register 7 contains the ASCII code for the letter A, what machine language instruction should be used to cause that letter to be printed at the printer?

store R7 -> B5 37B5

b. If the machine executes a million instructions per second, how many times can this character be sent to the printer in one second?

Machine executes 1,000,000 instructions per second

1 条 STORE 指令 times per second=1,000,000

c. If the printer is capable of printing five traditional pages of text per minute, will it be able to keep up with the characters being sent to it in (b)?

5×2000=10000 characters/min

10000/60≈167 characters/sec

167×8≈1336 bits/sec

No, the printer cannot keep up.

Because the CPU sends about 1,000,000 characters per second, while the printer can only handle about 167 characters per second.

2. Suppose that the hard disk on your personal computer rotates at 3000 revolutions a minute, that each track contains 16 sectors, and that each sector contains 1024 bytes. Approximately what communication rate is required between the disk drive and the disk controller if the controller is going to receive bits from the disk drive as they are read from the spinning disk?

16*3000*1024=49152000bytes/min

6,553,600 bits/sec ≈ 6.55 Mbps

3. Estimate how long it would take to transfer a 300 page novel encoded in 16 bit Unicode characters at a transfer rate of 54 Mbps.

300×2000=600,000 characters

600,000×16=9,600,000 bits

Time = total bits/rate =9,600,000/54,000,000 ≈0.178 seconds

2.6 Programming Data Manipulation

Python code → Interpreter → Machine instructions → CPU execution

Logic and Shift Operations

Just as Python uses the 0x prefix to specify values in hexadecimal, the 0b prefix can be used to specify values in binary1.

```
x    = 0b00110011
mask = 0b00001111
```

```
print(0b10011010 & 0b11001001)      #         10011010
                                    #     AND 11001001
                                    #         10001000

print(0b10011010 | 0b11001001)      #         10011010
                                    #     OR  11001001
                                    #         11011011

print(0b10011010 ^ 0b11001001)      #         10011010
                                    #     XOR 11001001
                                    #         01010011
```

```
print(0b00000101 ^ 0b00000100)      # Prints 5 XOR 4, which is 1
print(0b00000101 | 0b00000100)      # Prints 5 OR  4, which is 5
print(0b00000101 & 0b00000100)      # Prints 5 AND 4, which is 4
```

```
print(bin(0b10011010 & 0b11001001))   # Prints "0b10001000"
print(bin(0b10011010 | 0b11001001))   # Prints "0b11011011"
print(bin(0b10011010 ^ 0b11001001))   # Prints "0b1010011"
```

```
print(0b00111100 >> 2)   # Prints  "15", which is 0b00001111
print(0b00111100 << 2)   # Prints "240", which is 0b11110000
```

Control Structures

They are built on machine-level jump instructions.

```
if (water_temp > 140):
  print('Bath water too hot!')
```

```
while (n < 10):
  print(n)
  n = n + 1
```

Functions A function is a named sequence of operations that can be reused.

function call/calling A function call is the act of executing a function.

print("hello")

bin(5)

argument value An argument value is the input provided to a function.

print(5)

fruitful functions（有返回值函数） Fruitful functions return a value.

biggest = max(x, y, z)

void functions Void functions do not return a value.

print("hello")

procedures Procedures are another term for void functions.

Input and Output

Input and output (I/O) allow data to be transferred between the user and the computer.

Output

print("hello")

Input

echo = input("Enter: ")

print(echo * 3)

输入 "hi" → 输出 "hihihi"

Type Conversion（类型转换）

sideA = int(input("Length of side A? "))

sideA = int(input(...))

sideB = int(input(...))

Truncation Behavior（截断）

int("3.14") → 3

Marathon Training Assistant

Figure 2.15 Example marathon training data

Time Per Mile				Total Elapsed Time	
Minutes	Seconds	Miles	Speed (mph)	Minutes	Seconds
9	14	5	6.49819494584	46	10
8	0	3	7.5	24	0
7	45	6	7.74193548387	46	30
7	25	1	8.08988764044	7	25

```
# Marathon training assistant.
import math

# This function converts a number of minutes and seconds into just seconds.
def total_seconds(min, sec):
  return min * 60 + sec

# This function calculates a speed in miles per hour given
# a time (in seconds) to run a single mile.
def speed(time):
  return 3600 / time

# Prompt user for pace and mileage.
pace_minutes = int(input('Minutes per mile? '))
pace_seconds = int(input('Seconds per mile? '))
miles = int(input('Total miles? '))

# Calculate and print speed.
mph = speed(total_seconds(pace_minutes, pace_seconds))
print('Your speed is')
print(mph)

# Calculate elapsed time for planned workout.
total = miles * total_seconds(pace_minutes, pace_seconds)
elapsed_minutes = total // 60
elapsed_seconds = total % 60

print('Your total elapsed time is')
print(elapsed_minutes)
print(elapsed_seconds)
```

Body The body of a function contains the statements that define its behavior.

1. The hypotenuse example script truncates the sides to integers, but out puts a floating point number. Why? Adapt the script to output an integer.

```
# Calculates the hypotenuse of a right triangle import math
# Inputting the side lengths, with integer conversion
sideA = int(input('Length of side A? '))
sideB = int(input('Length of side B? '))
# Calculate third side via Pythagorean Theorem
hypotenuse = math.sqrt(sideA**2 + sideB**2)
print(hypotenuse)
```

➔ The math.sqrt() function always returns a floating-point number.

➔ print(int(hypotenuse))

2. Adapt the hypotenuse script to use floating point numbers as input, without truncating them. Which is more appropriate, the integer version from the previous question, or the floating point version?

the floating point version

```
import math
sideA = float(input('Length of side A? '))
sideB = float(input('Length of side B? '))
hypotenuse = math.sqrt(sideA**2 + sideB**2)
print(hypotenuse)
```

3. The Python built in function str() will convert a numerical argument into a character string representation, and the '+' can be used to concatenate strings together. Use these to modify the marathon script to produce cleaner output, for example:
Your speed is 7.74193548387 mph
Your total elapsed time is 46 minutes, 30 seconds

```
print('Your speed is '+str(mph)+ ' mph')
print('Your total elapsed time is '+str(elapsed_minutes) + ' minutes, '+str(elapsed_seconds) + ' seconds')
```

4. Use the Python built in bin() to write a script that reads a base 10 integers as input and outputs the corresponding binary representation of that integer in ones and zeros.

```
num = int(input('Enter a base 10 integer: '))
print(bin(num))
```

5. The XOR operation is often used both for efficiently calculating check sums (see Section 1.9) and encryption (see Section 4.5). Write a simple Python script that reads in a number and outputs that number XORed with a pattern of ones and zeros, such as 0x55555555. The same script will "encrypt" a number into a seemingly unrelated number, but when run again and given the encrypted number as input will return the original number.

```
num = int(input('Enter a number: '))
```

pattern = 0x55555555
result = num ^ pattern
print(result)

6. Explore some of the error conditions that you can create with unexpected inputs to the example scripts from this section.

What happens if you enter all zeros for the hypotenuse script or the marathon script?

1）hypotenuse = sqrt(0^2 + 0^2) = 0 √

2）speed = 3600 / 0 × ZeroDivisionError

What about negative numbers?

1）(-3)^2 = 9 √

2）Negative values lead to incorrect or meaningless results.

负速度（negative speed）

负时间（negative time）

The results are not physically meaningful.

Strings of characters instead of numbers?

1）sideA = input(...)

sideA ** 2

× TypeError

2）TypeError

ValueError

2.7 Other Architectures

Pipelining

Execution Speed 执行速度

Execution speed refers to how fast a single instruction is completed.

Throughput 吞吐量

Throughput refers to the total amount of work completed in a given time.

Pipelining 流水线

Pipelining is a technique that overlaps steps in the machine cycle.

取指 → 执行 → 取指 → 执行

取指(指令 1)
　　执行(指令 1)
取指(指令 2)
　　执行(指令 2)

The Multi-Core CPU

System-on-a-Chip (SoC)

A system-on-a-chip (SoC) integrates multiple components into a single chip.

The Multi-Core CPU

A multi-core CPU contains two or more CPUs on a single chip.

Multi-core → parallel execution using multiple CPUs

Multi-core CPUs improve throughput by executing multiple tasks at the same time.

Multiprocessor Machines

parallel processing

Parallel processing is the execution of multiple activities at the same time.

Pipelining is an initial step toward parallel processing.

Multiprocessor / Multi-core Machines（多处理器 / 多核）

True parallel processing requires more than one processing unit.

Multiprocessor or multi-core machines contain multiple CPUs.

Shared Memory Model（共享内存模型）

Multiple processors can share the same main memory.

Processors coordinate by writing and reading from shared memory.

MIMD (multiple-instruction stream, multiple-data stream)

MIMD means different processors execute different instructions on different data.

多个处理器执行不同指令，处理不同数据

SISD (single-instruction stream, single-data stream)

SISD represents traditional single-processor systems.

单指令流 + 单数据流（传统单核）

SIMD (single-instruction stream, multiple-data stream)

SIMD means one instruction is applied to multiple data items simultaneously.

一条指令同时作用于多个数据

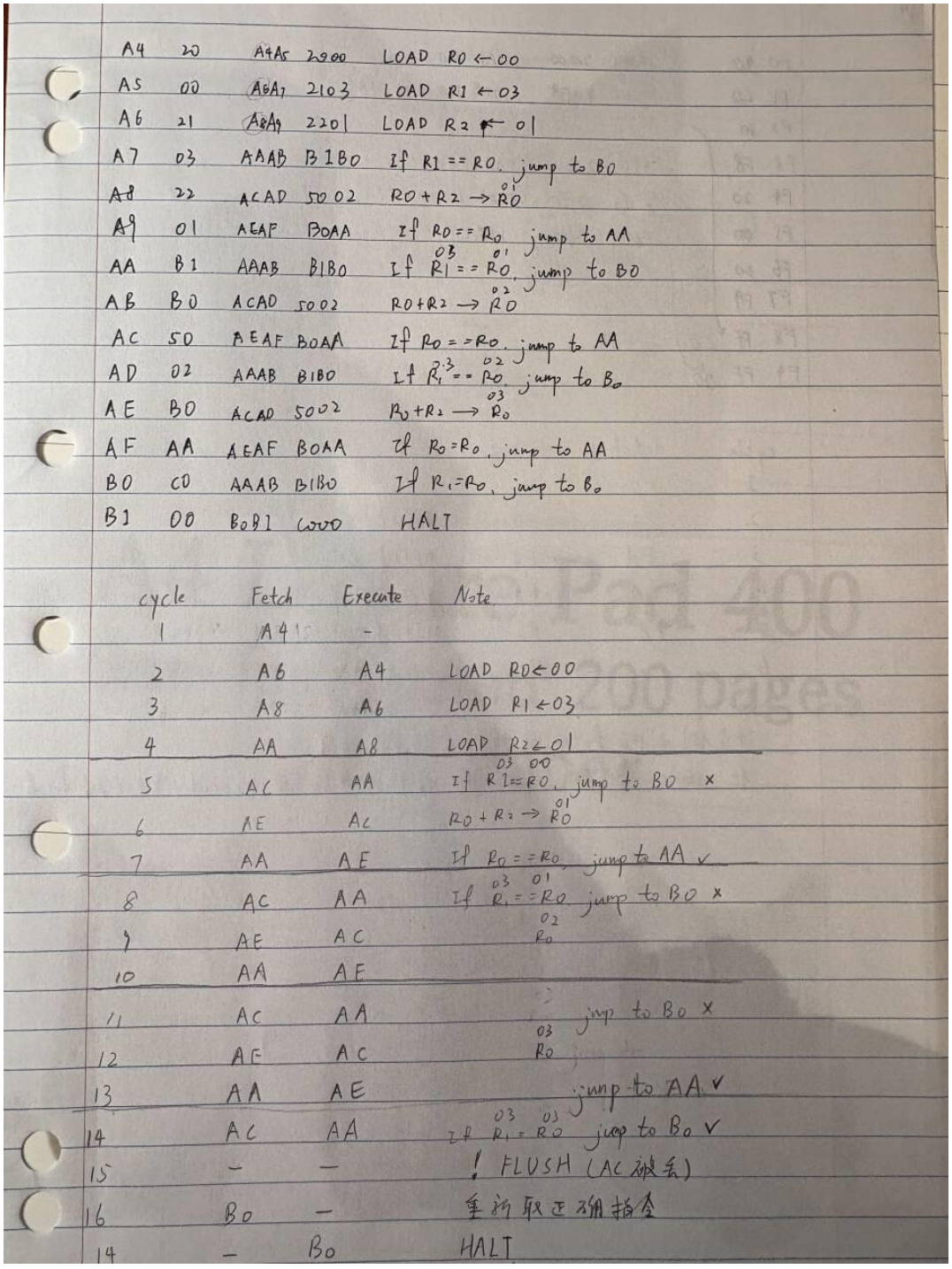
Distributed Systems（分布式并行）

Large systems can be built from smaller independent machines.

Each machine has its own CPU and memory.

Tasks are divided into subtasks and executed concurrently.

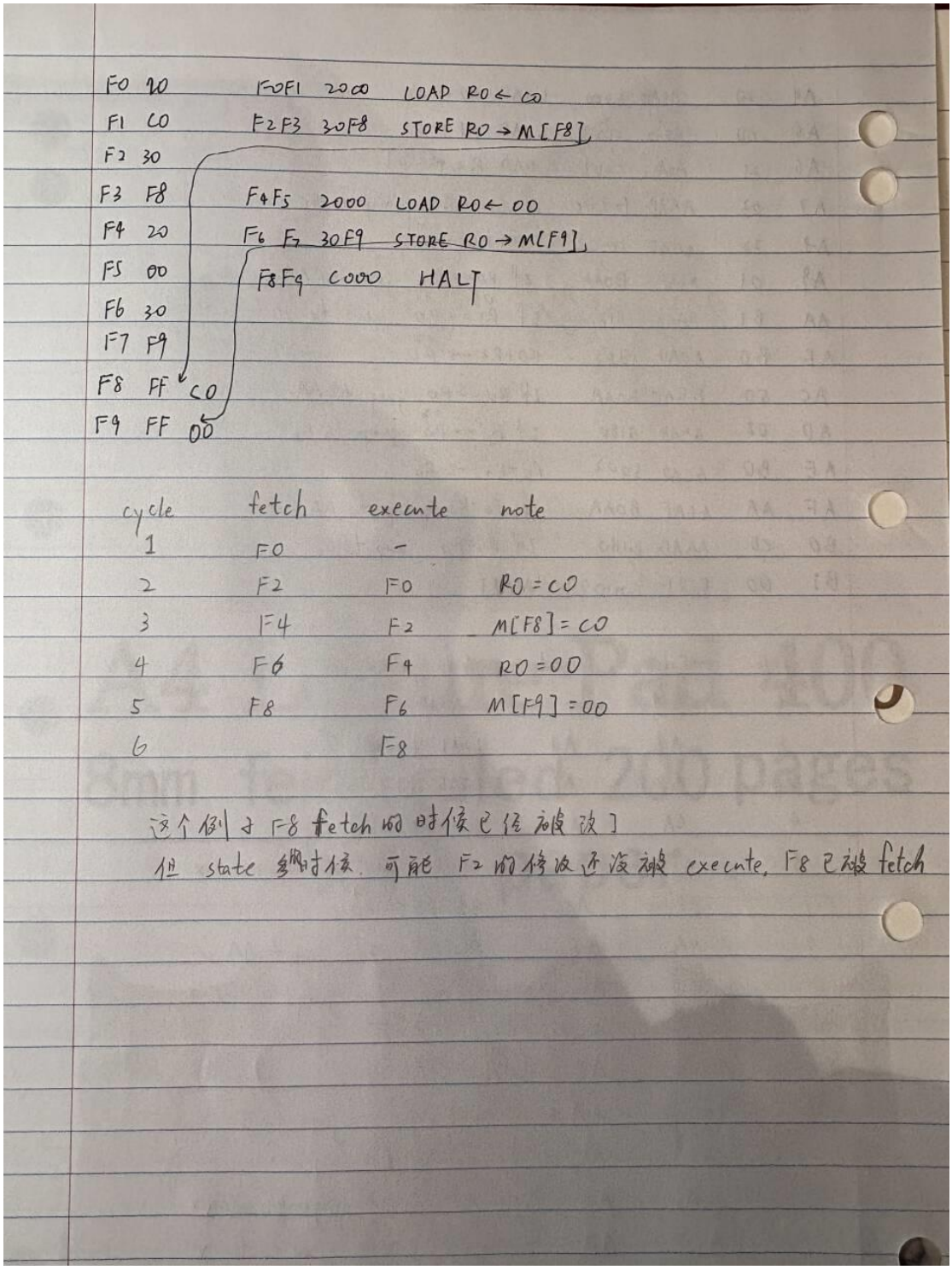
1. Referring back to question 3 of Section 2.3, if the machine used the pipe line technique discussed in the text, what will be in "the pipe" when the instruction at address AA is executed? Under what conditions would pipelining not prove beneficial at this point in the program?



During execution, instructions at AA, AC, and AE form a loop. In the pipeline, while one instruction is executed, the next is fetched. However, w hen the condition at AA becomes true, the instruction already fetched (AC) is discarded, and the pipeline is flushed before fetching the correct instruction at B0.

2. What conflicts must be resolved in running the program in question 4 of Section 2.3 on a pipeline machine?

The pipeline may fetch outdated instructions before the program modifies them.



3. Suppose there were two "central" processing units attached to the same memory and executing different programs. Furthermore, suppose that one of these processors needs to add one to the contents of a memory cell at roughly the same time that the other needs to subtract one from the same cell. (The net effect should be that the cell ends up with the same value with which it started.)

假设有两个"中央处理单元"（CPU）连接到同一块内存，并且各自执行不同的程序。

进一步假设，其中一个处理器需要在大致同一时间对某个内存单元的内容加 1，而另一个处理器需要对同一个内存单元的内容减 1。

a. Describe a sequence in which these activities would result in the cell ending up with a value one less than its starting value.

描述一种执行顺序，使得最终该内存单元的值比初始值少 1。

CPU1（+1）: READ → X

CPU2（-1）: READ → X

CPU1（+1）: MODIFY → X+1

CPU1（+1）: WRITE → M = X+1

CPU2（-1）: MODIFY → X-1

CPU2（-1）: WRITE → M = X-1 ← 覆盖前面的结果

b. Describe a sequence in which these activities would result in the cell ending up with a value one greater than its starting value.

描述一种执行顺序，使得最终该内存单元的值比初始值多 1。

CPU1（+1）: READ → X

CPU2（-1）: READ → X

CPU2（-1）: MODIFY → X-1

CPU2（-1）: WRITE → M = X-1

CPU1（+1）: MODIFY → X+1

CPU1（+1）: WRITE → M = X+1 ← 覆盖